

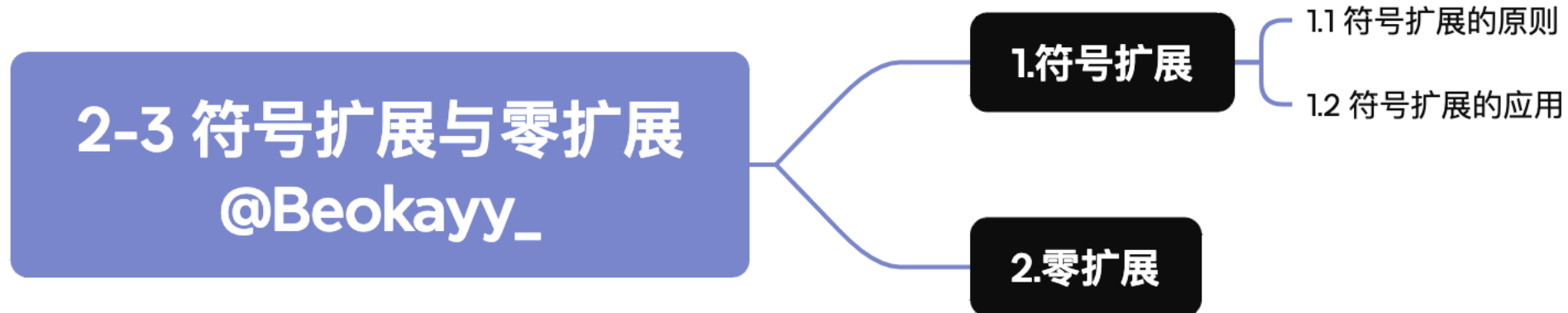
bok25

**基
础**

班、计算机组成原理



符号扩展与零扩展





1.符号扩展



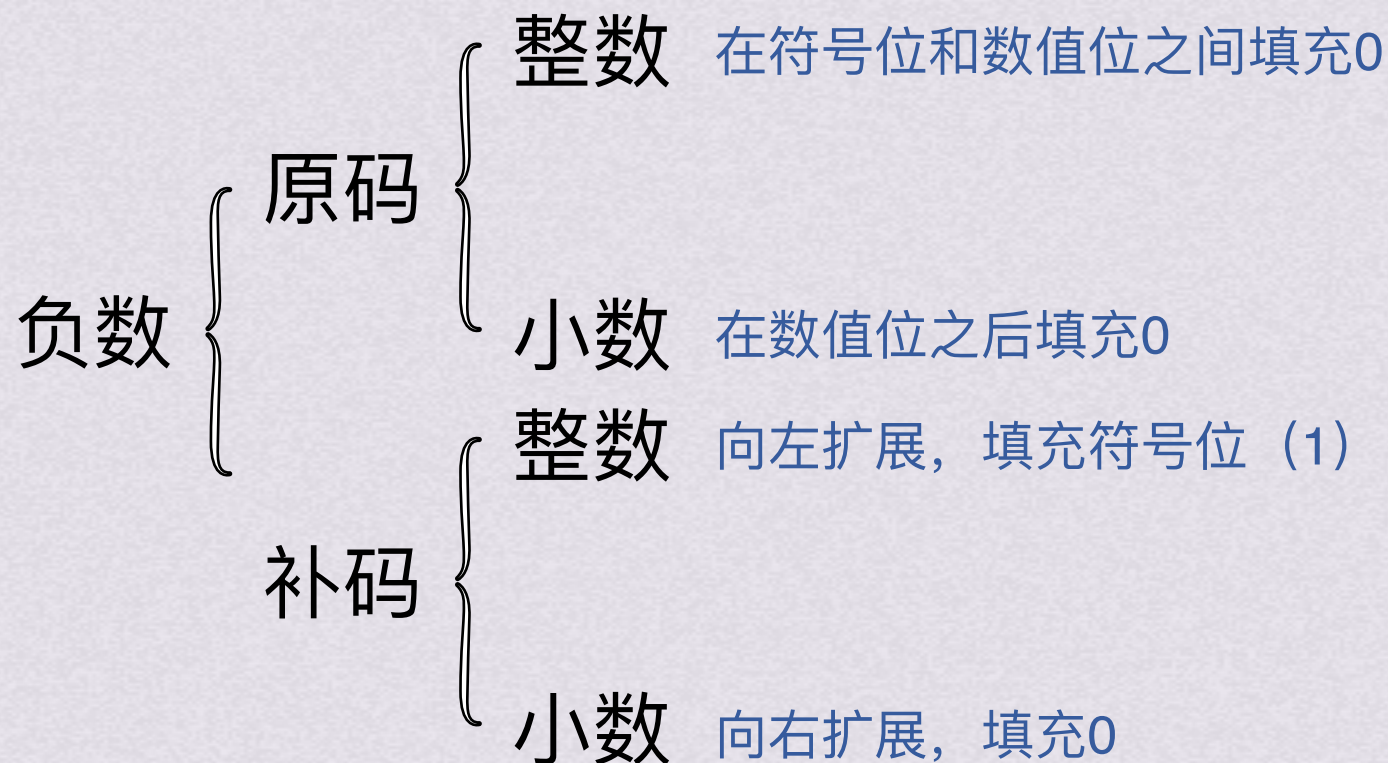
符号扩展与零扩展

1.符号扩展 1.1 符号扩展的原则

符号扩展

对象是带符号数

正数 符号位不变，所有扩展位都用0进行填充（整数向左，小数向右）





符号扩展与零扩展

1.符号扩展 1.1 符号扩展的原则

小试牛刀

请将以下机器数扩展到8位

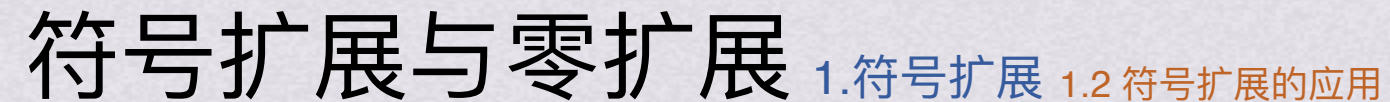
【原码】 1001

1000 0001

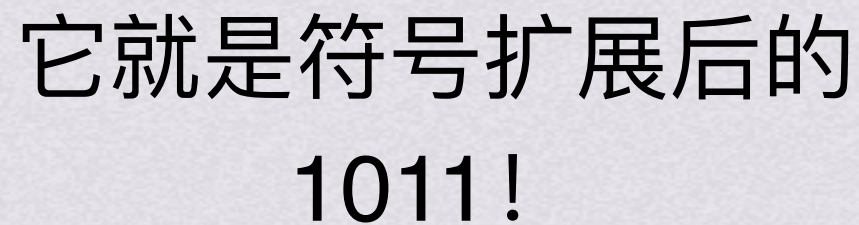
【补码】 1001

1111 1001





11111111111111111111111111111011





符号扩展与零扩展

1.符号扩展 1.2 符号扩展的应用

补码转真值



1011

$$-2^3 + 2^1 + 2^0$$



符号扩展与零扩展

1.符号扩展 1.2 符号扩展的应用

溢出判断

乘法计算后得到64位乘积

1111 1111 1111 1111 1111 1111 1111 1111 1110 1110 1000 1011 1010 0000 1111 0111

由于寄存器位数有限，只能保留低32位，那么该乘积是否溢出？

想一想符号扩展！





符号扩展与零扩展

1.符号扩展 1.2 符号扩展的应用

溢出判断

以int类型为例

乘法计算后得到64位乘积

1111 1111 1111 1111 1111 1111 1111 1111 1110 1110 1000 1011 1010 0000 1111 0111

由于寄存器位数有限，只能保留低32位，那么该乘积是否溢出？

高32位和第33位符号位相同，因此可看作符号扩展后的结果



符号扩展与零扩展

1.符号扩展 1.2 符号扩展的应用

溢出判断

以int类型为例

乘法计算后得到64位乘积

1111 1111 1111 1011 1111 1111 1111 1111 1110 1110 1000 1011 1010 0000 1111 0111

由于寄存器位数有限，只能保留低32位，那么该乘积是否溢出？

只有前13位相同，可看作符号扩展



2.零扩展



符号扩展与零扩展

2. 零扩展

零扩展

对象是无符号数

补0就完事了！